

# Software Design Document (SDD)

## ADC Driver — STM32F411RE

|                  |                                                           |
|------------------|-----------------------------------------------------------|
| <b>Project</b>   | Lab Assignment 03 — Microcontrollers and Embedded Systems |
| <b>Author</b>    | Luis Angel Lugo Muñiz (37915, B6)                         |
| <b>Course</b>    | MSE — Sixth Semester, CETYS Universidad Tijuana           |
| <b>Module</b>    | adc_driver                                                |
| <b>Target</b>    | STM32F411RE on NUCLEO-F411RE                              |
| <b>Toolchain</b> | arm-none-eabi-gcc + CMSIS, bare-metal                     |
| <b>Date</b>      | May 06, 2026                                              |

# 1. Introduction

## 1.1 Purpose

This document describes the design and architecture of the ADC driver module for the STM32F411RE microcontroller, based on the requirements defined in the SRS-ADC\_Driver. It provides details on data structures, register-level configuration, control flow, and the public API consumed by the higher-level sensor module. The intended audience is firmware developers integrating or maintaining this driver and reviewers verifying SRS compliance.

## 1.2 Scope

The ADC driver is a low-level software component that abstracts the STM32F4 ADC peripheral registers (CR1, CR2, SQRx, SMPRx, SR, DR) and exposes a clean, status-returning C API. The application uses this driver indirectly through the sensor module to convert the analog voltage at PA1 (ADC1\_IN1, the wiper of a potentiometer) into a 12-bit digital value that drives the duty cycle of a PWM output on PA8.

The driver supports single-channel, single-conversion, polling-mode operation on ADC1. Continuous mode, scan mode, DMA, and injected channels are out of scope for this lab; their hooks are documented as future enhancements (Section 5).

## 1.3 Definitions, Acronyms, and Abbreviations

| Term       | Definition                                                                 |
|------------|----------------------------------------------------------------------------|
| ADC        | Analog-to-Digital Converter peripheral                                     |
| ADON       | ADC ON bit (CR2 bit 0) — powers the peripheral                             |
| Channel    | Input multiplexer line connected to the ADC (channels 0–18 on F411RE)      |
| CMSIS      | Cortex Microcontroller Software Interface Standard (ARM)                   |
| DR         | Data Register — holds the most recent conversion result                    |
| EOC        | End Of Conversion flag (SR bit 1)                                          |
| OVR        | Overflow flag (SR bit 5) — set when DR is overwritten before being read    |
| Resolution | Number of bits in the digital result (6, 8, 10, or 12)                     |
| SQR        | Sequence Register — defines which channels are converted and in what order |
| SMPR       | Sample-time Register — programs the per-channel sampling time in cycles    |
| SWSTART    | Software Start bit (CR2 bit 30) — triggers a regular conversion            |
| Vref       | Reference voltage; on NUCLEO-F411RE Vref+ = 3.3 V                          |

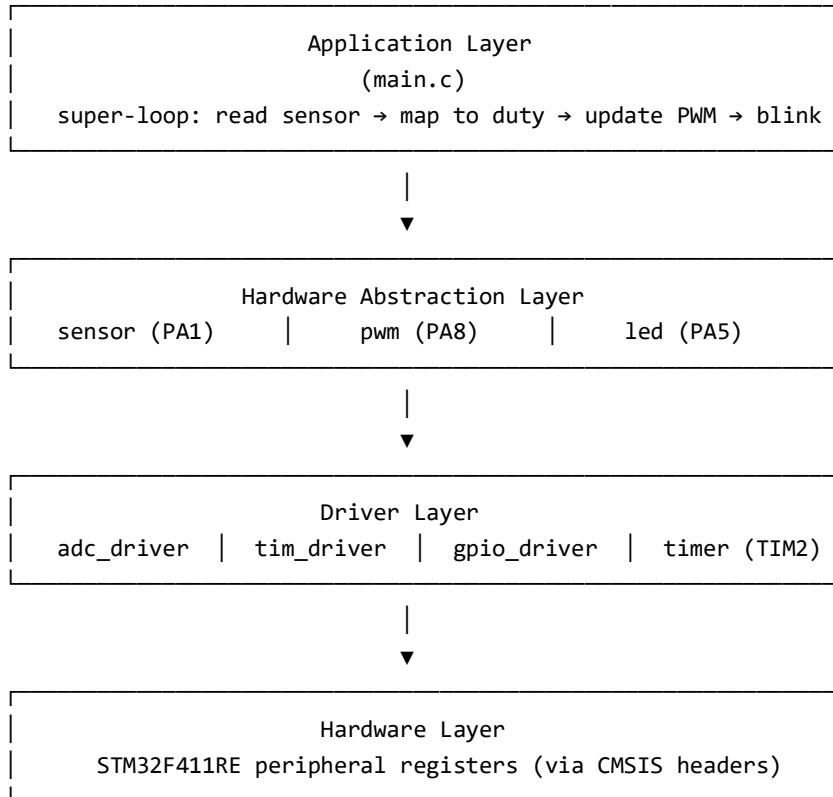
## 1.4 Requirements

All functional requirements specific to this module are allocated in SRS-ADC\_Driver.md. Section 4 of this SDD includes a traceability table mapping every implemented function to its corresponding SRS requirement (FR-1 through FR-9).

## 2. System Architecture

### 2.1 High-Level Design

The system follows a strict three-layer architecture. Each layer depends only on the layer immediately below it, which makes the driver portable and the application testable in isolation.



The ADC driver sits in the Driver Layer. It does not call any other driver — it only writes to the ADC1 register block exposed through CMSIS.

### 2.2 Module Interfaces

Public API: All functions defined in `adc_driver.h`. The seven public functions are: `adc_init`, `adc_set_channel`, `adc_enable`, `adc_disable`, `adc_start`, `adc_poll_eoc`, `adc_read`.

#### Dependencies:

- `stm32f411xe.h` (CMSIS device header) — provides `ADC_TypeDef`, `ADC1`, and bit-mask macros (`ADC_CR1_RES`, `ADC_CR2_ADON`, `ADC_SR_EOC`, etc.).
- `stdint.h`, `stddef.h` — fixed-width integer types and `NULL`.

#### Consumers:

- `sensor.c` — wraps `adc_driver` to provide a high-level read of the potentiometer voltage. The application never calls `adc_driver` functions directly.

#### Pre-conditions enforced outside the driver:

- The ADC1 peripheral clock must be enabled in RCC->APB2ENR before `adc_init()` is invoked. This is the responsibility of the sensor module (which also configures PA1 as analog through `gpio_driver`).
- `Vref+` must be supplied externally (provided by the NUCLEO-F411RE board at 3.3 V).

## 3. Detailed Design

### 3.1 Class Diagram

The driver is procedural C code, but the relationships between data types and functions can be expressed as a UML-style component view:

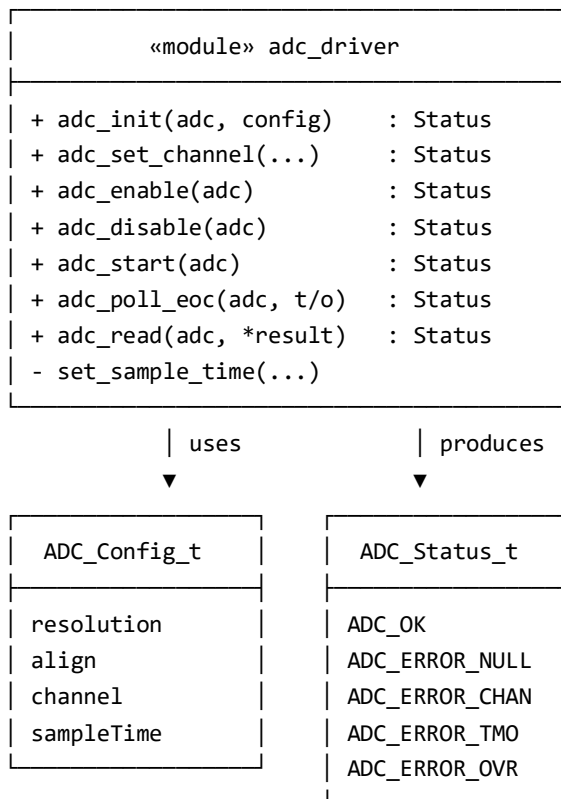
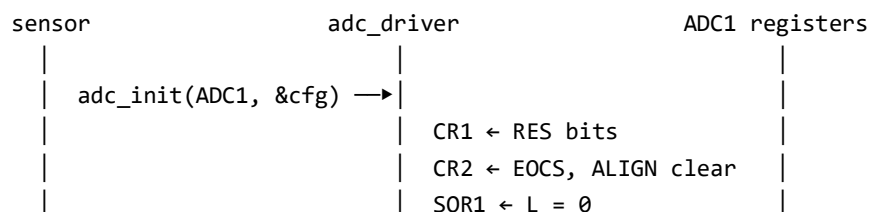


Figure 1. Component view of the ADC driver. Public functions take a pointer to `ADC_TypeDef` (CMSIS) and either a configuration struct or simple scalars, and always return an `ADC_Status_t` code.

### 3.2 Module Behavior

#### 3.2.1 Driver Initialization (Sequence)

The standard call sequence performed by the sensor module on system startup:



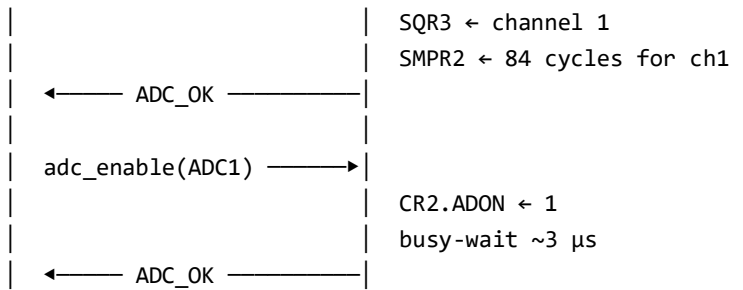


Figure 2. Sequence — driver initialization. Note that `adc_init()` does not power on the peripheral; this is a deliberate split (see Section 3.3) so the caller can configure all channels before any conversion is possible.

### 3.2.2 Single Conversion (Activity)

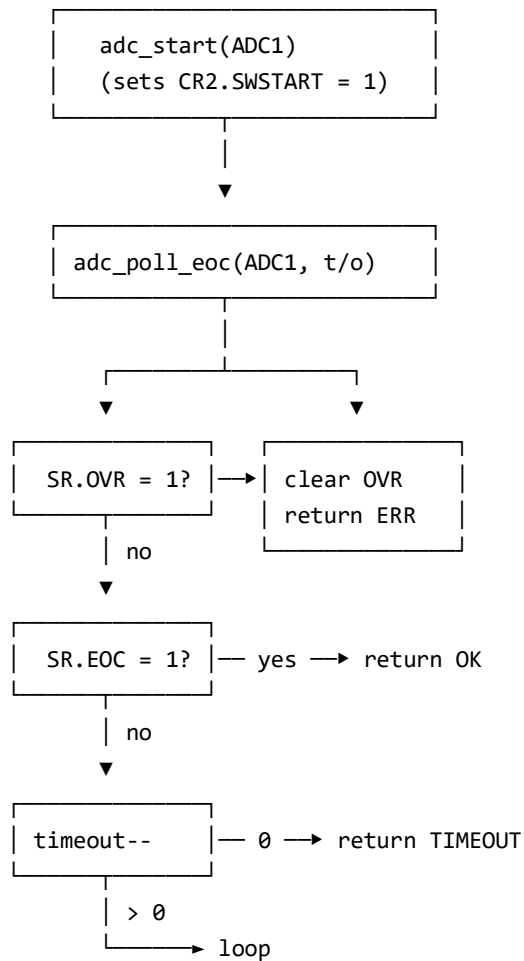
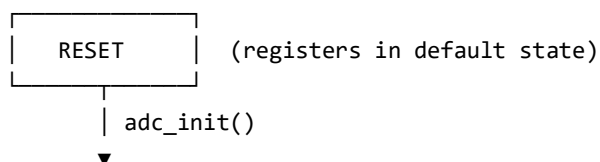


Figure 3. Activity — polled conversion with overrun and timeout guards.

### 3.2.3 Driver State Diagram



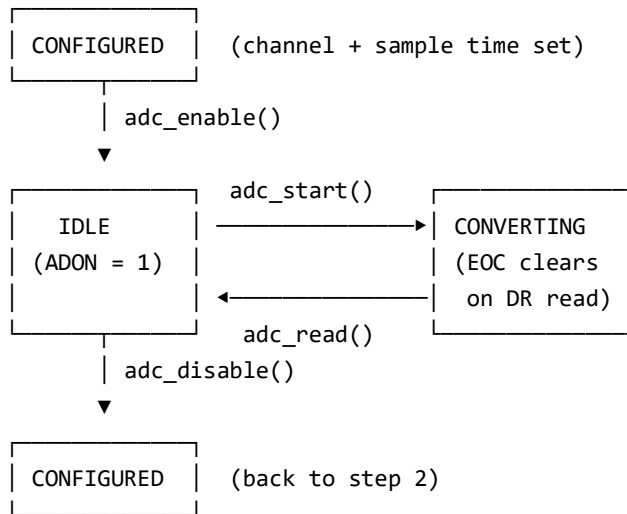


Figure 4. State diagram of the ADC peripheral as managed by this driver.

### 3.3 ADC Driver — Design Rationale

#### Why a configuration struct (ADC\_Config\_t)?

The four parameters that must be set together (resolution, alignment, channel, sample time) are passed as a struct so adding a new field later (e.g., continuousMode) does not break existing callers. The struct is taken const \* so the caller can declare it static const in flash and avoid stack copies.

#### Why split `adc_init()` from `adc_enable()`?

The reference manual (RM0383, §11.3.1) requires ADON = 0 while CR1/CR2/SQRx are written. Splitting the API forces the caller into the correct sequence: configure first, power on second. It also lets the application disable the ADC for low-power modes without re-configuring on wake-up.

#### Why polling instead of EOC interrupt?

A 12-bit conversion with a 84-cycle sample time at the default 16 MHz ADCCLK takes roughly 6 μs. The `sensor_read` call site is the slowest stage of the super-loop already, and the simplicity of polling makes the driver easier to reason about for the lab's modular-design rubric. Interrupt-driven conversion is listed as a future enhancement.

#### Why explicit overrun handling?

The STM32F4 ADC is a fast peripheral and OVR can fire if a higher-priority interrupt delays the DR read across two conversions. Returning `ADC_ERROR_OVERRUN` instead of silently giving stale data lets the upper layer log the event or retry.

#### Why a timeout in `adc_poll_eoc()`?

A driver that can busy-wait forever is a single point of failure for the entire firmware. The timeout (`ADC_TIMEOUT_CYCLES = 100,000`) is generous enough (~6 ms at 16 MHz) that legitimate conversions never trip it, but short enough that a wedged ADC will surface as a fault inside one super-loop iteration instead of locking the system.

#### Sample time choice (84 cycles).

The sensor module selects ADC\_SMP\_84\_CYCLES because the potentiometer wiper has non-trivial source impedance ( $\approx 5\text{ k}\Omega$  at midpoint). Per the F411RE datasheet (DS9716, Table 67), 84 cycles is the recommended minimum for sources up to  $\sim 10\text{ k}\Omega$  at 12-bit resolution.

## 4. Application Programming Interfaces

### 4.1 Data Types and Macros

#### ADC\_Resolution\_t (enum)

Selects the ADC conversion resolution by writing CR1.RES[25:24].

| Member        | Value | Range  |
|---------------|-------|--------|
| ADC_RES_12BIT | 0x00  | 0–4095 |
| ADC_RES_10BIT | 0x01  | 0–1023 |
| ADC_RES_8BIT  | 0x02  | 0–255  |
| ADC_RES_6BIT  | 0x03  | 0–63   |

#### ADC\_Align\_t (enum)

Selects right- or left-alignment in the data register (CR2.ALIGN[11]).

| Member          | Value | Notes                                                    |
|-----------------|-------|----------------------------------------------------------|
| ADC_ALIGN_RIGHT | 0x00  | Default; result occupies LSBs of DR                      |
| ADC_ALIGN_LEFT  | 0x01  | Result occupies MSBs of DR (for 12-bit fixed-point math) |

#### ADC\_SampleTime\_t (enum)

Per-channel sample time programmed in SMPRx (3-bit field, eight values).

| Member            | Cycles | Member             | Cycles |
|-------------------|--------|--------------------|--------|
| ADC_SMP_3_CYCLES  | 3      | ADC_SMP_84_CYCLES  | 84     |
| ADC_SMP_15_CYCLES | 15     | ADC_SMP_112_CYCLES | 112    |
| ADC_SMP_28_CYCLES | 28     | ADC_SMP_144_CYCLES | 144    |
| ADC_SMP_56_CYCLES | 56     | ADC_SMP_480_CYCLES | 480    |

#### ADC\_Config\_t (struct)

Aggregates all parameters passed to adc\_init().

| Member     | Type             | Notes                 |
|------------|------------------|-----------------------|
| resolution | ADC_Resolution_t | Conversion resolution |
| align      | ADC_Align_t      | Data alignment in DR  |

| Member     | Type             | Notes                                  |
|------------|------------------|----------------------------------------|
| channel    | uint8_t          | ADC input channel, 0–18                |
| sampleTime | ADC_SampleTime_t | Sample time for the configured channel |

## Macros

| Macro                 | Value   | Purpose                                 |
|-----------------------|---------|-----------------------------------------|
| ADC_STAB_DELAY_CYCLES | 48U     | NOPs after ADON to ensure stabilization |
| ADC_TIMEOUT_CYCLES    | 100000U | Default poll budget for adc_poll_eoc()  |
| ADC_MAX_CHANNEL       | 18U     | Highest valid channel on STM32F411RE    |

## 4.2 Error Codes

All public functions return ADC\_Status\_t.

| Code              | Value | Notes                                        |
|-------------------|-------|----------------------------------------------|
| ADC_OK            | 0     | Operation completed successfully             |
| ADC_ERROR_NULL    | 1     | NULL pointer passed as argument (NFR-2)      |
| ADC_ERROR_CHANNEL | 2     | Channel number > 18 (NFR-2)                  |
| ADC_ERROR_TIMEOUT | 3     | EOC did not assert within the timeout budget |
| ADC_ERROR_OVERRUN | 4     | SR.OVR was set; data integrity lost          |

## 4.3 Function Definitions

### adc\_init

|                    |                                                                                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Function name      | adc_init                                                                                                                                                                                |
| Syntax             | ADC_Status_t adc_init(ADC_TypeDef *adc, const ADC_Config_t *config)                                                                                                                     |
| Parameters (in)    | adc — ADC peripheral pointer (e.g., ADC1). config — pointer to a populated configuration struct.                                                                                        |
| Parameters (inout) | None                                                                                                                                                                                    |
| Parameters (out)   | None                                                                                                                                                                                    |
| Return value       | ADC_Status_t — ADC_OK, ADC_ERROR_NULL, or ADC_ERROR_CHANNEL.                                                                                                                            |
| Description        | Configures CR1 (resolution), CR2 (alignment, single conversion, EOCS=1), SQR1 (sequence length = 1), and delegates channel + sample time to adc_set_channel. Does not power on the ADC. |
| Constraints        | The ADC peripheral clock must already be enabled (RCC->APB2ENR). Function does not validate the resolution or alignment fields beyond the type system.                                  |



|                      |                                                   |
|----------------------|---------------------------------------------------|
| <b>Available via</b> | adc_driver.h                                      |
| <b>SRS trace</b>     | FR-1 (init), FR-2 (peripheral enable preparation) |

### adc\_set\_channel

|                        |                                                                                                                                                        |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function name</b>   | adc_set_channel                                                                                                                                        |
| <b>Syntax</b>          | ADC_Status_t adc_set_channel(ADC_TypeDef *adc, uint8_t channel, ADC_SampleTime_t smp)                                                                  |
| <b>Parameters (in)</b> | adc — ADC peripheral pointer. channel — 0 to 18. smp — sample time selection.                                                                          |
| <b>Return value</b>    | ADC_OK, ADC_ERROR_NULL, or ADC_ERROR_CHANNEL.                                                                                                          |
| <b>Description</b>     | Writes the channel number into SQR3.SQ1[4:0] and the 3-bit sample time into the corresponding field of SMPR2 (channels 0–9) or SMPR1 (channels 10–18). |
| <b>Constraints</b>     | Only configures the first conversion slot; multi-channel sequences are not supported.                                                                  |
| <b>Available via</b>   | adc_driver.h                                                                                                                                           |
| <b>SRS trace</b>       | FR-3 (channel selection)                                                                                                                               |

### adc\_enable

|                        |                                                                                                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function name</b>   | adc_enable                                                                                                                                                        |
| <b>Syntax</b>          | ADC_Status_t adc_enable(ADC_TypeDef *adc)                                                                                                                         |
| <b>Parameters (in)</b> | adc — ADC peripheral pointer.                                                                                                                                     |
| <b>Return value</b>    | ADC_OK or ADC_ERROR_NULL.                                                                                                                                         |
| <b>Description</b>     | Sets CR2.ADON to power on the peripheral, then burns ADC_STAB_DELAY_CYCLES NOPs to satisfy the minimum stabilization time before any conversion can be triggered. |
| <b>Constraints</b>     | Busy-waits with NOPs (no SysTick dependency). Delay is calibrated for ~16 MHz core; faster cores will over-wait but remain correct.                               |
| <b>Available via</b>   | adc_driver.h                                                                                                                                                      |
| <b>SRS trace</b>       | FR-4 (enable peripheral)                                                                                                                                          |

### adc\_disable

|                        |                                            |
|------------------------|--------------------------------------------|
| <b>Function name</b>   | adc_disable                                |
| <b>Syntax</b>          | ADC_Status_t adc_disable(ADC_TypeDef *adc) |
| <b>Parameters (in)</b> | adc — ADC peripheral pointer.              |
| <b>Return value</b>    | ADC_OK or ADC_ERROR_NULL.                  |

|                      |                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| <b>Description</b>   | Clears CR2.ADON; further adc_start calls have no effect until adc_enable is invoked again. |
| <b>Constraints</b>   | Does not flush in-flight conversions; the caller must ensure no adc_start is pending.      |
| <b>Available via</b> | adc_driver.h                                                                               |
| <b>SRS trace</b>     | FR-5 (disable peripheral; covers the inverse of FR-4)                                      |

## adc\_start

|                        |                                                                                                                                                                  |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function name</b>   | adc_start                                                                                                                                                        |
| <b>Syntax</b>          | ADC_Status_t adc_start(ADC_TypeDef *adc)                                                                                                                         |
| <b>Parameters (in)</b> | adc — ADC peripheral pointer.                                                                                                                                    |
| <b>Return value</b>    | ADC_OK or ADC_ERROR_NULL.                                                                                                                                        |
| <b>Description</b>     | Sets CR2.SWSTART, triggering a single regular conversion on the channel previously configured. Hardware clears SWSTART automatically when the conversion begins. |
| <b>Constraints</b>     | Requires adc_enable to have been called and the stabilization delay to have elapsed.                                                                             |
| <b>Available via</b>   | adc_driver.h                                                                                                                                                     |
| <b>SRS trace</b>       | FR-6 (start single conversion via software trigger)                                                                                                              |

## adc\_poll\_eoc

|                        |                                                                                                                                                                                                                          |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function name</b>   | adc_poll_eoc                                                                                                                                                                                                             |
| <b>Syntax</b>          | ADC_Status_t adc_poll_eoc(ADC_TypeDef *adc, uint32_t timeout)                                                                                                                                                            |
| <b>Parameters (in)</b> | adc — ADC peripheral pointer. timeout — maximum loop iterations to wait.                                                                                                                                                 |
| <b>Return value</b>    | ADC_OK, ADC_ERROR_NULL, ADC_ERROR_TIMEOUT, or ADC_ERROR_OVERRUN.                                                                                                                                                         |
| <b>Description</b>     | Busy-loops on SR until either EOC asserts (success), OVR asserts (data lost — flag is cleared and error returned), or the timeout counter reaches zero.                                                                  |
| <b>Constraints</b>     | The timeout unit is loop iterations, not microseconds — actual elapsed time depends on core clock and compiler optimizations. The conservative default macro ADC_TIMEOUT_CYCLES corresponds to ~6 ms at 16 MHz with -O0. |
| <b>Available via</b>   | adc_driver.h                                                                                                                                                                                                             |
| <b>SRS trace</b>       | FR-7 (wait for EOC); NFR-2 (error handling)                                                                                                                                                                              |

## adc\_read

|                      |          |
|----------------------|----------|
| <b>Function name</b> | adc_read |
|----------------------|----------|

|                         |                                                                                                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Syntax</b>           | ADC_Status_t adc_read(ADC_TypeDef *adc, uint16_t *result)                                                                                                             |
| <b>Parameters (in)</b>  | adc — ADC peripheral pointer.                                                                                                                                         |
| <b>Parameters (out)</b> | result — pointer to a 16-bit variable receiving the raw conversion value.                                                                                             |
| <b>Return value</b>     | ADC_OK or ADC_ERROR_NULL.                                                                                                                                             |
| <b>Description</b>      | Reads DR (which automatically clears SR.EOC in hardware) and stores the lower 16 bits in *result. Caller is responsible for interpreting the alignment and bit width. |
| <b>Constraints</b>      | Does not check that EOC was actually set — the caller must invoke adc_poll_eoc first or accept that *result may contain a stale value.                                |
| <b>Available via</b>    | adc_driver.h                                                                                                                                                          |
| <b>SRS trace</b>        | FR-8 (read regular conversion result)                                                                                                                                 |

## 4.4 Internal Function Definitions

### set\_sample\_time (static helper)

|                        |                                                                                                                                                                                                                                                                              |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function name</b>   | set_sample_time                                                                                                                                                                                                                                                              |
| <b>Syntax</b>          | static void set_sample_time(ADC_TypeDef *adc, uint8_t channel, ADC_SampleTime_t smp)                                                                                                                                                                                         |
| <b>Parameters (in)</b> | adc — ADC peripheral pointer. channel — 0–18. smp — sample time selection.                                                                                                                                                                                                   |
| <b>Return value</b>    | void                                                                                                                                                                                                                                                                         |
| <b>Description</b>     | Writes the 3-bit sample-time field into SMPR1 or SMPR2 depending on the channel. For channels 0–9 the bit-field offset is channel * 3 in SMPR2; for channels 10–18 the offset is (channel - 10) * 3 in SMPR1. The existing field is cleared first to allow re-configuration. |
| <b>Constraints</b>     | Internal helper — does not validate inputs (callers adc_init and adc_set_channel already do). Not exported in the header.                                                                                                                                                    |

## Traceability Summary

| SRS Requirement | Description                   | Implementation                              |
|-----------------|-------------------------------|---------------------------------------------|
| FR-1            | Initialize the ADC subsystem  | adc_init                                    |
| FR-2            | Activate the ADC peripheral   | adc_enable                                  |
| FR-3            | Select valid regular channels | adc_set_channel                             |
| FR-4            | Configure injected channels   | Not implemented (out of scope, see §5)      |
| FR-5            | Start single conversion       | adc_start (preceded by adc_init/adc_enable) |
| FR-6            | Start continuous conversion   | Not implemented (out of scope, see §5)      |

| SRS Requirement | Description                       | Implementation                                                      |
|-----------------|-----------------------------------|---------------------------------------------------------------------|
| FR-7            | Start injected conversion         | Not implemented (out of scope, see §5)                              |
| FR-8            | Read regular conversion result    | adc_read (with adc_poll_eoc for synchronization)                    |
| FR-9            | Read injected channel result      | Not implemented (out of scope, see §5)                              |
| NFR-1           | Lightweight and efficient         | Polling, no allocations, ~250 bytes of .text                        |
| NFR-2           | Error handling for invalid inputs | ADC_Status_t returned by every function                             |
| NFR-3           | Real-time performance             | Bounded adc_poll_eoc timeout; total conversion $\leq 6 \mu\text{s}$ |

## 5. Future Enhancements

The current implementation covers the lab requirements (regular conversion on a single channel, polling mode). The following extensions are reasonable next steps and are explicitly listed because the SRS scopes them as future work:

1. Injected channel support (FR-4, FR-7, FR-9). Add `adc_set_injected_channel`, `adc_start_injected`, and `adc_read_injected_data` wrapping JSQRx, JSWSTART, and JDRx. Useful for sampling synchronized to a timer event (e.g., motor-control current sensing).
2. Continuous mode (FR-6). Set `CR2.CONT` and replace the EOC poll with a circular-buffer DMA stream, so the application can read the latest sample in  $O(1)$  without triggering each conversion.
3. DMA support. Configure DMA2 stream 0/4 for ADC1 to remove the CPU from the data path entirely. This is the natural extension when the application grows to multiple channels.
4. Multi-channel scan. Support sequences of length  $> 1$  by accepting an `ADC_Channel_t[]` and writing `SQR1/SQR2/SQR3` with the full sequence and length.
5. Calibration helper. STM32F4 ADCs do not require user calibration, but future ports to families that do (G0, L4+) would benefit from an `adc_calibrate` function.
6. Interrupt-driven conversion. Enable `EOCIE` in `CR1` and provide a registered callback so the application can sleep between conversions in a low-power mode.
7. Resolution-aware read. `adc_read` could shift the result based on the configured alignment/resolution so the caller never sees raw register layout. Currently this is left to the consumer (`sensor.c`) for transparency.